Observer

Observer · Fan out video.published to audit+UI · Build with Waqar

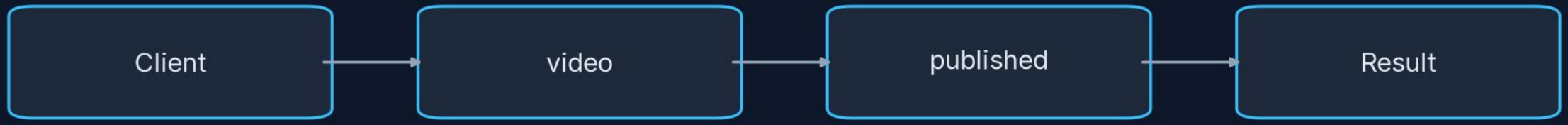
Today's Hook

- Problem: Fan out video.published to audit+UI
- Pattern: Observer — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Fan out video.published to audit+UI
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Observer: Fan out video.published to audit+UI

Real-World Scenario

- Running example: Fan out video.published to audit+UI
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.observer;

import com.buildwithwaqar.patterns.gof.behavioral.observer.EventBus;
import java.util.ArrayList;
/** Problem: fan out domain events to audit and UI without hardwiring. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Observer problem: event fan-out ===");
    var bus = new EventBus<String>();
    var audit = new ArrayList<String>();
    try (var s = bus.subscribe(e -> audit.add(e))) {
      bus.publish("video.published");
    }
    System.out.println("audit=" + audit);
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.observer;

import java.util.ArrayList;
import java.util.List;
import java.util.function.Consumer;

public final class EventBus<T> {
    private final List<Consumer<T>> subscribers = new ArrayList<>();

    public AutoCloseable subscribe(Consumer<T> listener) {
        subscribers.add(listener);
        return () -> subscribers.remove(listener);
    }

    public void publish(T event) {
        for (var s : List.copyOf(subscribers)) s.accept(event);
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.behavioral.observer;

import com.buildwithwaqar.patterns.gof.behavioral.observer.EventBus;
import java.util.ArrayList;
/** Problem: fan out domain events to audit and UI without hardwiring. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Observer problem: event fan-out ===");
    var bus = new EventBus<String>();
    var audit = new ArrayList<String>();
    try (var s = bus.subscribe(e -> audit.add(e))) {
      bus.publish("video.published");
    }
    System.out.println("audit=" + audit);
  }
}
```

Walkthrough Output

```
=== Observer problem: event fan-out ===
audit=[video.published]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Observer.

Related Patterns

- Contrast Observer with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Observer → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.